IITM PRAVARTAK
CATALYSING INNOVATION

Professional Certificate Programme in Agentic AI and Applications

Memory & Knowledge Retrieval in Agents with MCP

Revisit the Learning So Far

Let’s take a quick look at the key ideas and concepts you have explored in your learning journey.

- How goals are defined, interpreted and executed within autonomous AI agents
- Real-world applications that demonstrate agent-based systems in practice
- The differences between hierarchical planning and reactive planning approaches
- Planning strategies applied and adapted in real operational scenarios
- Autonomous agents making decisions and acting independently within defined environments
- Multi-agent planning strategies that enable coordination and collaboration between agents

Topics Covered

- Introduction to Memory, MCP & A2A
- Why MCP is the New Universal Agent Tooling Interface
- Agent-to-Agent (A2A) Communication and Multi-Agent Systems

Introduction to Memory, MCP & A2A

(Topic 1 of 3)

Introduction to Memory, MCP & A2A

Modern AI is evolving from single LLM prompts to coordinated systems of reasoning, where multiple intelligent components perceive, reason, act and communicate

The shift drives three core capabilities:

- Memory
- Model Context Protocol (MCP)
- Agent-to-Agent (A2A) communication

Introduction to Memory, MCP & A2A

Industry trend

OpenAI, Anthropic, Google, Stripe, Notion and Databricks are adopting memory systems, MCP and multi-agent protocols for agent-based architectures.

Why this matters

Businesses require AI that can perform multi-step, multi-agent workflows rather than simple chat responses.

Introduction to Memory, MCP & A2A

Examples

- Customer-support agents retrieving CRM history (Memory)
- Loan assistants interacting with risk engines via standardised interfaces (MCP)
- Developer agents collaborating and handing off tasks through structured exchanges (A2A)

These technologies form the foundation of modern agentic systems and are essential for building reliable, scalable AI applications.

Memory : Short-Term v. Long-Term

Memory is what transforms an LLM from a “text generator” into a “context-aware agent.”

- Modern AI systems need memory beyond single-turn interactions
- Memory enables context retention, preference learning and consistent decision-making

Memory : Short-Term vs Long-Term

Short-term memory

- Temporary
- Lives only during the current task
- Helps the agent follow the conversation and reasoning steps

Long-term memory

- Persistent
- Stored across sessions
- Helps the agent remember user info, past interactions and knowledge

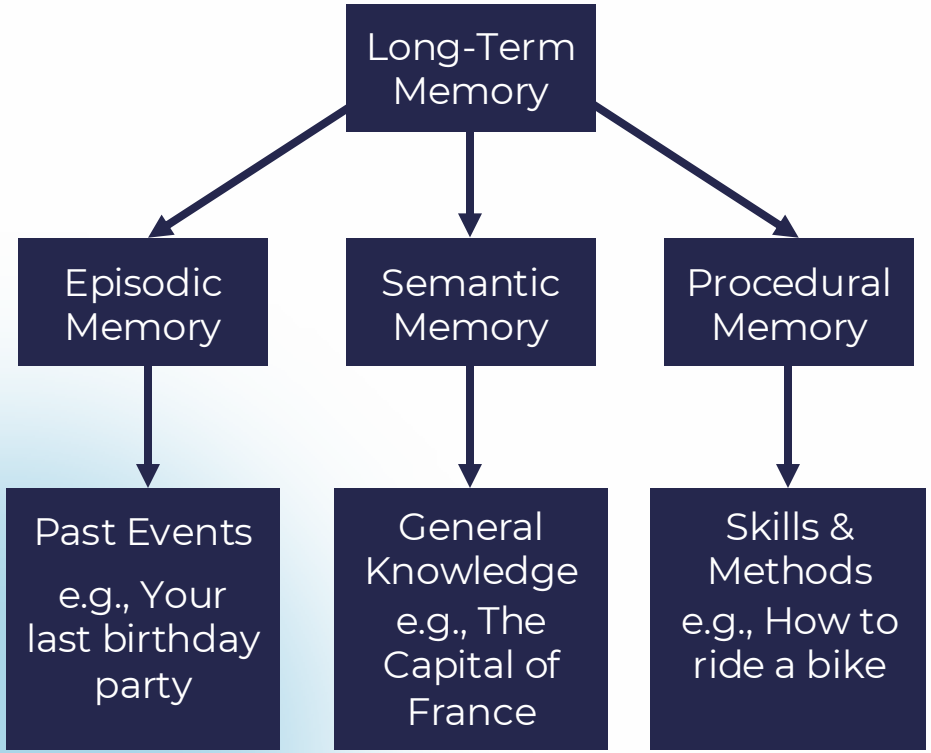
What is Short-Term Memory in AI Agents?

- **Like computer RAM:** Holds information temporarily during tasks or conversations
- **Limited capacity:** ~5-9 items, lasting seconds to minutes
- **Real-time decisions:** Chatbots track queries, cars process sensors

Challenge: STM overwrites quickly, losing context and creating fragmented interactions

What is Long-Term Memory in AI Agents

Like a hard drive: Stores vast knowledge and experiences over days, months or years



Types:

- **Episodic:** Past events
- **Semantic:** General knowledge
- **Procedural:** Skills and methods

Why Both Memories Matter

- **STM: Immediate context:** Provides context for ongoing tasks
- **LTM: expertise & personality:** Builds knowledge over time

Why Both Memories Matter

Modern AI Frameworks

Tools & techniques

- LangGraph, Redis
- Memory checkpoints
- Smart summarisation

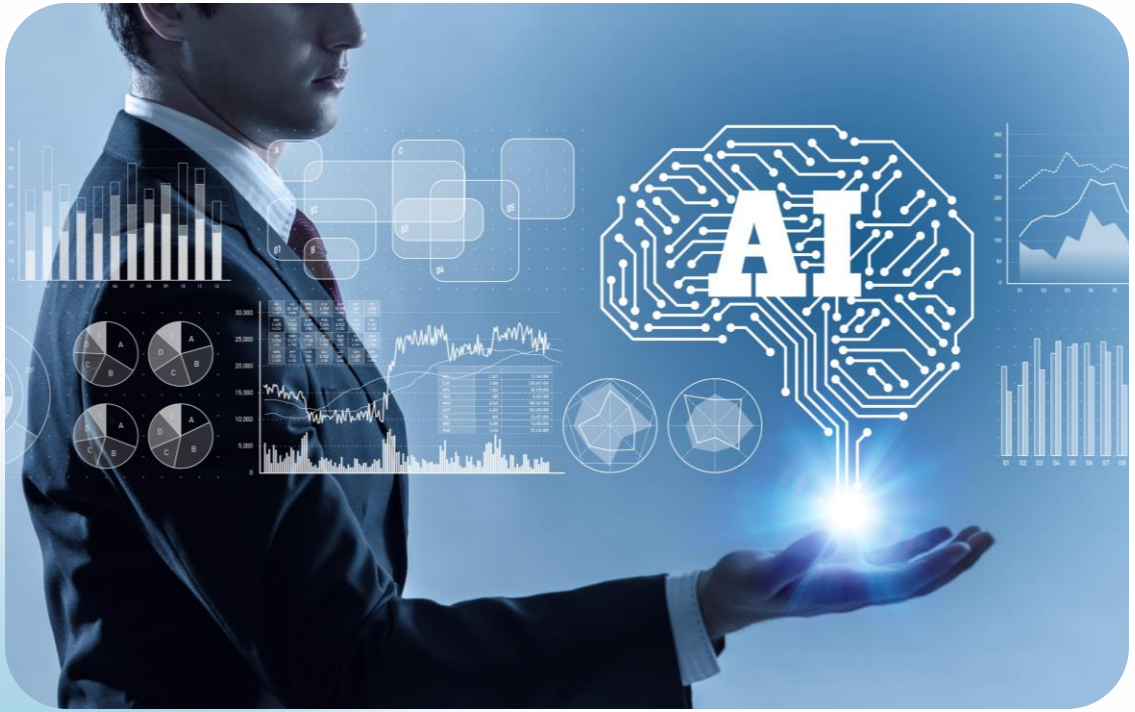
Balance performance

- Optimise memory use while maintaining continuity

The future

- Enable smarter, adaptive AI agents
- Improve user experience and productivity

Memory Storage Technique: Embedding Vectors



- AI systems store long-term memory as embedding vectors
- These vectors capture meaning and enable efficient retrieval

Memory Storage Technique: Embedding Vectors

How it works

- Convert text → embedding vector
- Store vectors in a vector database (Pinecone, FAISS, Chroma, etc.)
- Retrieve the most relevant memory using similarity search
- Feed retrieved context back to the model

Why embeddings

- Capture semantic meaning, not just keywords
- Enable fast and accurate memory lookup
- Support scalable long-term memory for agent workflows

Model Context Protocol (MCP)

As AI systems gain memory and long-term context, they need standardised access to tools, data and external systems

Allows models to securely call tools, APIs and data sources

Defines a consistent protocol for exchanging context

Ensures agents can retrieve memory, perform actions and update information safely

Model Context Protocol (MCP)

Why it Matters

- Avoids custom integrations for every model
- Enables scalable, plug-and-play connections
- Supports agent workflows that require reliable access to external knowledge and actions

Note: Memory stores what the agent knows. MCP defines how the agent accesses and uses that knowledge.

Why MCP is the New Universal Agent Tooling Interface

Discover how the Model Context Protocol is revolutionising the way AI agents connect with tools, data and services across the modern enterprise landscape.

The Integration Bottleneck Before MCP

Custom integration hell

AI agents required one-off integrations for each tool or API, increasing maintenance overhead

Fragmented connections

Non-standardised, brittle connections limited scalability and innovation

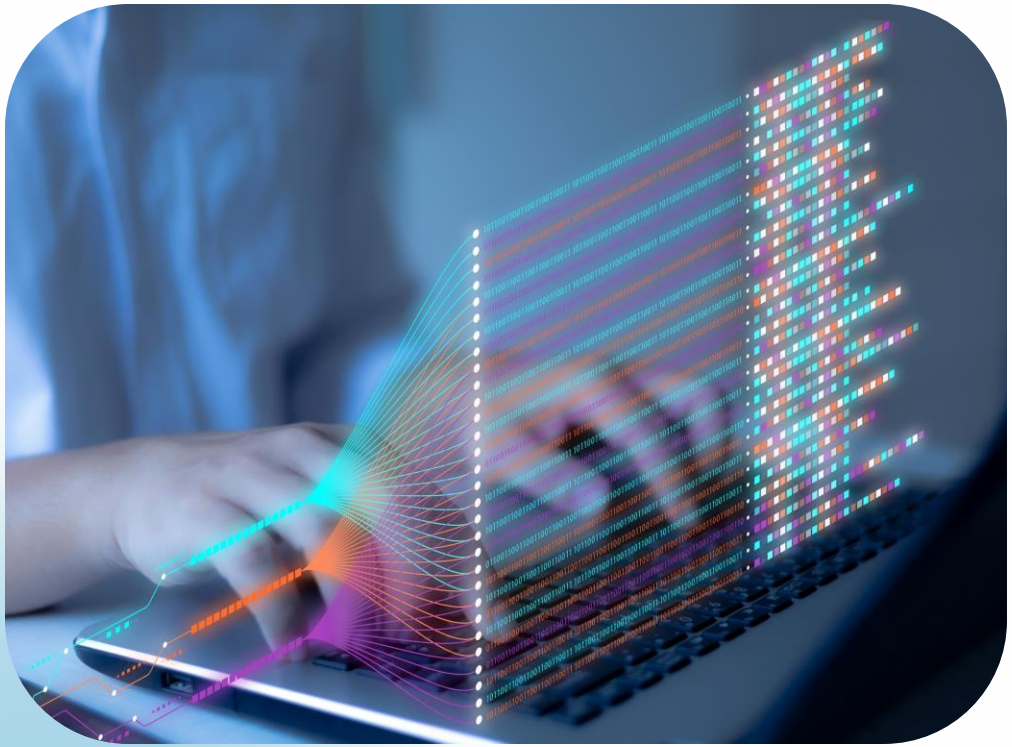
The MCP solution

An open standard launched by Anthropic in November 2024 to unify tool access across the AI ecosystem

Explosive growth

Thousands of MCP servers and SDKs now enable seamless agent interaction with systems

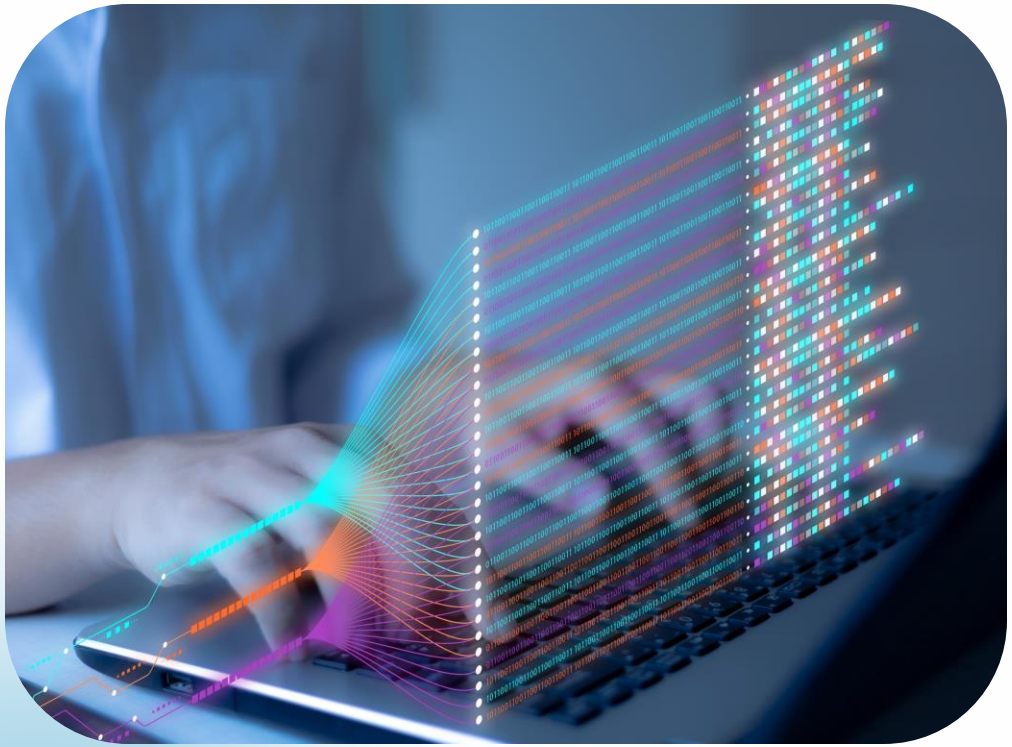
MCP: The Protocol that Connects them Cleanly



Model Context Protocol (MCP) is a standard for AI systems to safely connect to:

- Tools
- APIs
- Databases
- Files
- Business systems
- External services

MCP: The Protocol that Connects them Cleanly



- Standardises how AI agents request information from external systems to respond
- Gives protocols that tells:

  - ✓ How to request
  - ✓ How to respond
  - ✓ How to pass arguments
  - ✓ How to handle errors
  - ✓ How to return results

This makes agents safer, consistent, predictable and easy to integrate with enterprise systems.

Anatomy of MCP: The Three Pillars

Tool server

Hosts and exposes external tools, APIs, or data sources via the MCP protocol, acting as the universal gateway

Client interface

The AI agent or application that discovers, authenticates with and calls MCP servers to access tools

Resource providers: Underlying

systems such as databases, apps and services that power the tools and provide functionality

These components form a universal, standardised communication protocol that fundamentally transforms AI tooling architecture.

Tool Server: The Universal Adapter

What it does: Provides an intelligent translator between AI agents and external tools

Real-world example: Exposes document retrieval, creation and management via a Google Drive MCP server

Today's ecosystem: Supports thousands of MCP servers across CRMs, file systems, databases and communication platforms

Value proposition: Enables rapid integration without custom code, reducing development time

Client Interface: The Agent’s Control Panel

Discovery & authentication

Manages automatic discovery of available tools and handles secure authentication flows within AI agents

Standardised communication

Processes tool definitions, parameters and responses using the standardised JSON-RPC 2.0 format for consistency

Efficient execution

Supports optimised code execution paths to significantly reduce token usage and minimise latency in agent operations

Complex orchestration

Empowers agents to orchestrate sophisticated, multi-step workflows seamlessly across multiple MCP servers and tools

Resources Providers: The Data & Action Engines

CRM systems

Salesforce, HubSpot and other customer relationship platforms provide live data and automation capabilities

Productivity tools

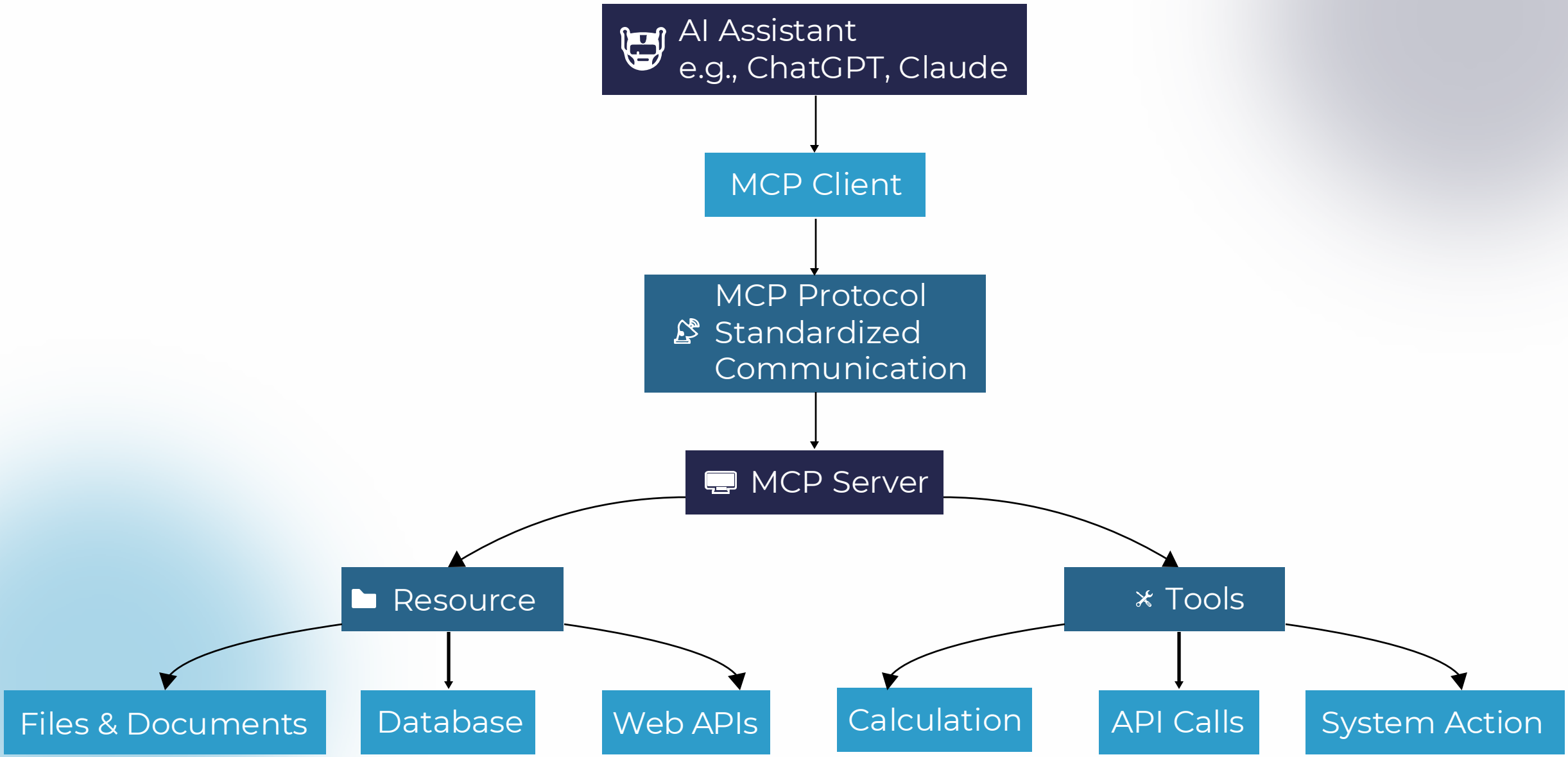
Notion, Slack and collaboration platforms enable agents to manage workflows and communications

Data systems

Internal databases, data warehouses and analytics platforms provide the foundation for intelligent decision-making

MCP abstracts complex APIs, allowing agents to interact with any system without bespoke integration code, while enabling secure, real-time access at enterprise scale.

Resources Providers: The Data & Action Engines



When to Use MCP vs Direct API Calls

Choose MCP when

- Needing scalable, multi-tool orchestration across diverse systems and platforms
- Prioritising rapid integration and reduced maintenance
- Requiring security, governance and standardised access controls
- Building enterprise-grade agentic automation with complex workflows

Direct API calls may suffice for

- Simple, single-tool use cases with minimal complexity
- Quick prototyping and proof-of-concept work
- Legacy systems where MCP servers don't yet exist

MCP excels in enterprise environments that require sophisticated automation, robust security and large-scale multi-tool orchestration.

Security & Sandboxing Considerations

Robust authentication

Supports OAuth 2.0 and token-based authentication, ensuring only authorised agents access sensitive tools and data

Execution isolation

Isolates tool execution through sandboxed MCP servers to prevent unauthorised data leaks and contain security threats

Centralised governance

Enables unified auditing, real-time monitoring and compliance enforcement across all integrations

Reduced risk profile

Delivers consistent, standardised security controls that lower risk compared to ad-hoc API integrations

Real- World Impact: MCP in Action

100+

Tools connected

Enterprise AI assistants now connect to hundreds of tools simultaneously via MCP

80%

Time savings

Reduction in engineering overhead for deploying AI-powered automation

5x

Faster deployment

Speed increase in launching new agent capabilities compared to custom integrations

Real- World Impact: MCP in Action

Transformative Use Cases

Automated multi-step workflows combining CRM updates, document retrieval and email drafting in seconds

Enhanced agent collaboration creating compound intelligence effects across organisational boundaries

Rapid deployment of sophisticated automation with dramatically less engineering investment

MCP adoption is accelerating rapidly, with major LLM providers, enterprise platforms and development tools integrating native support.

MCP Real- World Examples

An AI assistant can check Google calendar and automatically schedule or reschedule meetings.

A developer-facing AI can Read Figma designs and generate UI code.

A business chatbot can query multiple databases to analyse sales performance and customer trends.

An operations agent can read and update Excel or Sheets files to generate reports and track inventory.

A customer-support AI can retrieve CRM data from tools such as HubSpot or Salesforce to answer customer-specific queries.

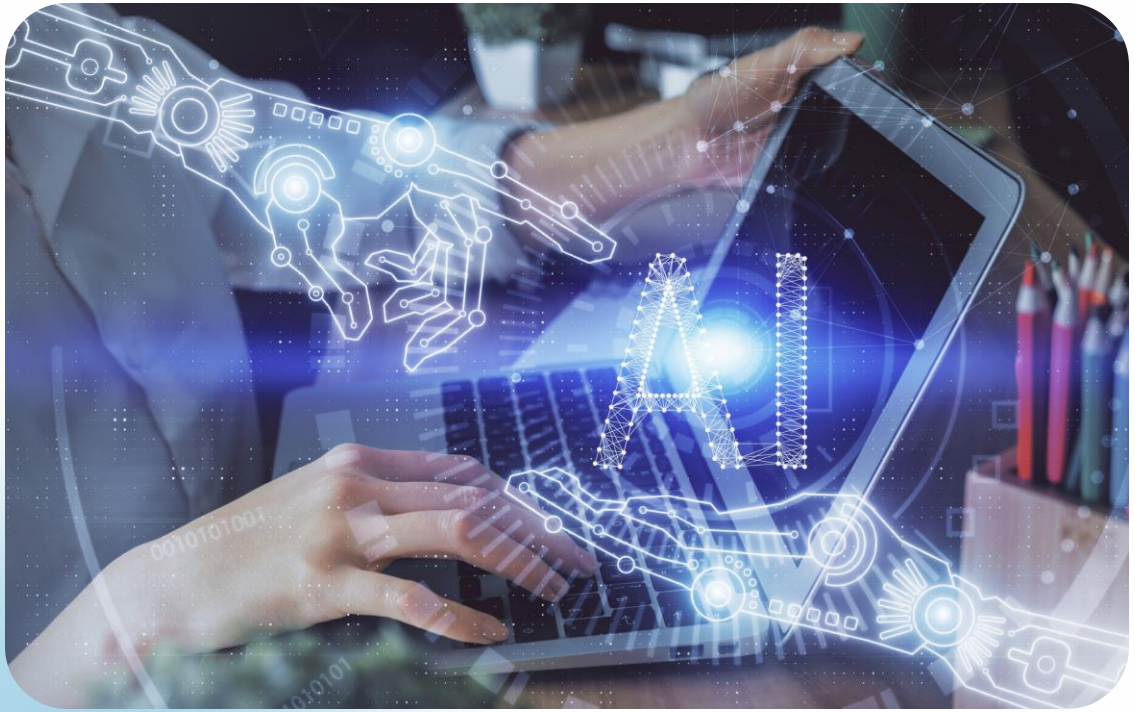
A workflow automation agent can run shell commands and trigger APIs to deploy builds, run scripts and manage servers.

Demo: Build an MCP server with N8N

Agent-to-Agent (A2A) Communication and Multi-Agent Systems

(Topic 3 of 3)

Agent-to-Agent (A2A) Communication



- Enables multi-agent collaboration as models gain memory and standardised tool access
- Provides structured agent coordination, task handoffs and context sharing

Agent-to-Agent (A2A) Communication

What A2A enables

- Agents passing outputs, plans and decisions to one another
- Specialised agents collaborating on multi-step workflows
- Standardised message formats preventing ambiguity

Why it matters

- Real-world tasks require multiple skills
- A2A enables reliable teamwork between agents built on the memory and MCP foundation

Note: MCP lets an agent access tools and data. A2A lets multiple agents communicate and orchestrate tasks together.

Why A2A Protocols are Essential

- When multiple agents operate without structured protocols, systems become unreliable and unpredictable
- When agent communication is unmanaged, system behaviour becomes inconsistent and ineffective

Why A2A Protocols Are Essential

Message confusion

Agents misinterpret instructions or duplicate efforts when messages lack clear structure and intent definitions

Infinite loops

Agents enter recursive communication cycles without proper termination conditions, consuming resources indefinitely

Context loss

Critical information gets dropped between handoffs, forcing agents to restart processes or make decisions with incomplete data

Protocols solve these issues by establishing standardised communication patterns, clear message contracts and enforceable coordination rules.

Core Components of A2A Protocols

Message format

JSON schemas with required fields for role identification, intent declaration, payload data and routing metadata

Input/output contracts

Explicit type definitions and validation rules for expected inputs and conformant outputs

Allowed actions

Whitelist of permissible operations, including API calls, data transformations and agent invocations

Context rules

Specifications for information sharing between agents, context accumulation and state reset or pruning

Types of Agent Interactions

Parallel

Multiple agents work independently on different subtasks simultaneously, then results are aggregated or merged

Negotiation-based

Agents follow consensus protocols, proposing solutions and iterating until agreement or fallback

Sequential

Linear handoff pattern where each agent completes its task before passing control to the next agent in the chain

Hierarchical

Planner agent decomposes work and delegates to specialised worker agents, maintaining orchestration and coordination

Message Passing Patterns

- A2A protocols govern message flow through agent systems
- A2A patterns ensure predictable behaviour and enable complex workflows

Message Passing Patterns

Task assignment
Coordinator broadcasts work packages with clear objectives, constraints and success criteria to available agents

Plan → execute → validate
Planners create strategies, executors implement them and validators verify correctness before proceeding

Error routing
Failed operations trigger fallback agents or escalation paths based on error type, severity and retry policies

Confidence escalation
Low-confidence responses route automatically to higher-capability agents or human review based on thresholds

Coordination Mechanisms

Turn-taking protocols

Enforce explicit token-passing so only one agent acts at a time, preventing race conditions and preserving coherence

Supervision models

Coordinate agents through centralised supervisors or decentralised, peer-based self-organisation

Role-based routing

Route messages using role tags to dynamically match tasks with agent capabilities

Memory architecture

Manage shared memory for global context while preserving private memory for agent-specific reasoning

Demo: Multi Agents System (Agent 2 Agent Protocol)

Safety and Reliability in A2A

Loop prevention
Implements maximum iteration counts, cycle detection and timeouts to prevent infinite recursion and resource exhaustion

Consistency checks
Validates outputs from multiple agents and triggers reconciliation or escalation to resolve conflicts

Action guardrails
Enforces multi-agent approval, human-in-the-loop confirmation, or constraint verification before executing sensitive operations

Transparent reasoning
Provides chain-of-thought prompting in messages to enable auditability, debugging and trust

Standard Agent Protocol Flow

Planner agent

Analyses the task, decomposes it into discrete steps and creates an execution strategy with clear objectives for each phase

Worker agent

Receives plan, executes assigned steps using available tools and APIs and returns results with execution metadata

Reviewer agent

Validates worker output against plan requirements, checks for errors or inconsistencies and approves or requests revisions

Composer agent

Aggregates validated results, formats final output and ensures response meets original task requirements and quality standards

This canonical pattern appears across major multi-agent frameworks and provides a reliable architecture for complex tasks.

Key Takeaways

Structure prevents chaos: Protocols transform unpredictable agent interactions into reliable, auditable systems with clear accountability

Composable patterns: Standard coordination mechanisms enable complex behaviours through a combination of simple, well-defined primitives

Structure prevents chaos: Protocols transform unpredictable agent interactions into reliable, auditable systems with clear accountability

A2A protocols are the foundation for building production-grade multi-agent systems that scale reliably.

Q & A

Thank you